

ControlPlane

Detailed Solution Document

An assurance gateway that decides how much checking each AI answer is worth, enforces a safety floor the budget cannot override, and writes every decision into a ledger that can be recomputed from its own arithmetic.

92.8%

of the protection of checking every answer, for 10.0% of the compute

1,500

held out interactions behind every figure in this document

0.008

to 0.043 expected calibration error by route

1,500 of 1,500

decisions in one verified hash chain

Team
BurningShadows

Repository
github.com/Advait-Pathak9222/BurningShadows

Reproduce
`./run_submission.sh`

Evidence
Offline. No API key, no network, no GPU

Contents

01	Executive summary	What this is and what it demonstrates
02	The problem	Why fixed verification wastes money in both directions
03	Solution approach	Verification as a spending decision
04	Architecture	Four planes and a learning loop
05	End to end request flow	One held out request, stage by stage
06	Core decision logic	Tiers, floor, allocation, review, effects
07	Audit and reproducibility	The ledger and the regeneration path
08	Implementation	What is built, and where it lives
09	Evaluation and evidence	Measured results on held out traffic
10	External benchmarks	Five public corpora, including one we failed
11	Running the system	Commands, console and live prototype
12	Deployment approach	Where each tier belongs in production
13	Current validation boundaries	What is proven and what is not
14	Conclusion	What we would do next

01 Executive summary

Enterprises are putting language models in front of customers, staff and money. Something has to check what those models say before it reaches a person or moves a rupee. ControlPlane is the layer that decides how much checking each answer is worth.

Today there are two common answers and both waste money. Checking every answer with a strong model is thorough and can cost more than generating the answer did. Checking a fixed sample is cheap and blind, because it spends the same effort on a question about opening hours as on an instruction to transfer funds. Neither asks what it would cost if this particular answer were wrong.

ControlPlane scores each answer on five harm axes, calibrates those scores into probabilities, multiplies them by what that harm costs on that route, and buys a check only where the loss prevented beats the price of the check. Underneath sits a per route release floor selected on held out data, which makes a check mandatory regardless of budget. Above it sits a reviewer queue that treats human attention as the scarce resource it actually is. Everything lands in a hash chained ledger.

92.8%

of the benefit of checking everything, for 10.0% of the compute

81 to 98%

of the assurance bill is human review, not compute

1.59x

more expected loss served from the same reviewer hours

0%

of verdicts flip across a 0.25x to 4x swing in prices

What this document demonstrates

Every number here is produced by a committed command, written into a tracked file, and checked for byte identical reproduction by `./run_submission.sh`. The evaluation runs offline against a seeded provider and lexical detector stubs. That is deliberate. Freezing detector quality is what makes the measured gain attributable to allocation rather than to a vendor model, and it is why the numbers reproduce exactly on a reviewer machine.

Three results are reported because they went against us. Allocation is not universally better than a well tuned fixed rate policy. Our score on the Aegis benchmark sits below the weakest published system. The learning loop currently refuses to release a refit on all three routes because the label supply is too thin. Each is set out in full rather than omitted.

02 The problem

Verification of AI output is treated as a fixed rule when it is really a spending decision under a budget.

Checking everything costs more than it saves on most traffic

Assurance tiers differ enormously in price. In our cost model the cheapest tier is a rules pass at 2 paise and 4 milliseconds. The most expensive is a language model judge at ₹3.20 and 900 milliseconds. That is **160 times the cost and 225 times the delay**. Running the judge on all traffic adds 900 milliseconds to every answer, which is a product regression on its own, and on 2 million interactions a year it costs ₹64 lakh before a single reviewer is paid.

Checking a fixed sample is blind to what is at stake

A one in ten sample treats every request as equally important. It cannot tell that a wrong opening time costs an apology while a wrong payment instruction costs money, a regulator and an incident review. Our policy files price that difference explicitly. On the finance route a data leak is priced at ₹50,000 and an exfiltration attempt at ₹55,000, against ₹1,000 for a hallucination on the internal knowledge base route.

The bill is mostly people, and almost nobody optimises that half

Whatever the automated checks escalate, a person has to read. On our configured prices one completed review costs ₹120 while the dearest automated check costs ₹3.20, so a review is **37.5 times an automated check**. Measured across the budget grid, human attention is 81 to 98 paise in every assurance rupee.

Compute budget	Automated checking	Human review	Attention share	Cases raised
10%	₹480.98	₹19,920	97.6%	246
25%	₹1,098.58	₹19,920	94.8%	327
40%	₹1,920.18	₹19,920	91.2%	344
60%	₹2,812.84	₹19,920	87.6%	406
80%	₹3,839.64	₹19,920	83.8%	484
100%	₹4,800.00	₹19,920	80.6%	569

Attention share by compute budget, over 1,500 held out interactions at fixed reviewer capacity. Source [docs/results/results.json](#).

Takeaway. Raising the compute budget raises the number of cases needing a person, because more checking finds more to escalate. Reviewer capacity is fixed, so the queue saturates and what differs between policies is which cases get served. That makes serving order a first class design problem rather than an implementation detail.

Nobody can show their working afterwards

When a regulator or an internal auditor asks why a specific answer was released, most guardrail stacks can produce a log line. They cannot produce the arithmetic. ControlPlane records the calibrated risk vector, the consequence table, the threshold, the shadow price and the policy hash for every decision, so the decision can be recomputed rather than merely recalled.

03 Solution approach

Treat every check as a purchase. Estimate what it would prevent, compare that with what it costs, and buy it only when the first number is larger. Then put a floor underneath so that safety is never a purely economic question.

```
expected loss = calibrated risk × consequence  
check when expected loss × catch rate > (1 + shadow price) × check cost
```

Every term in that rule is a real quantity with a source. **Calibrated risk** comes from detector scores passed through an isotonic map fitted per route and per axis, so the number behaves like a probability. **Consequence** comes from a versioned policy file that finance and risk own. **Catch rate** is what a tier actually finds, measured from reviewer labels rather than assumed. **Shadow price** is the live price of budget, which rises when spend runs ahead of the allowance.

Five ideas hold the design together

Harm is a vector, not a label

An answer can leak data and be wrong at the same time. Scores are kept on five axes through the whole pipeline, because collapsing them early destroys the information the consequence table needs.

Evidence regime decides what is knowable

Whether an answer can be checked at all depends on what it was given. Grounded, estimable and unverifiable are recorded separately, and an unverifiable answer with meaningful risk is withheld rather than guessed at.

The floor is not negotiable

A release threshold is selected per route on a held out fold so the rate of harm escaping unchecked stays under a stated bound. The budget can slow everything else down. It cannot switch this off.

Attention is the scarce resource

Reviewer minutes cost 37.5 times an automated check and cannot be bought at short notice, so the queue is scheduled by value per minute against a deadline.

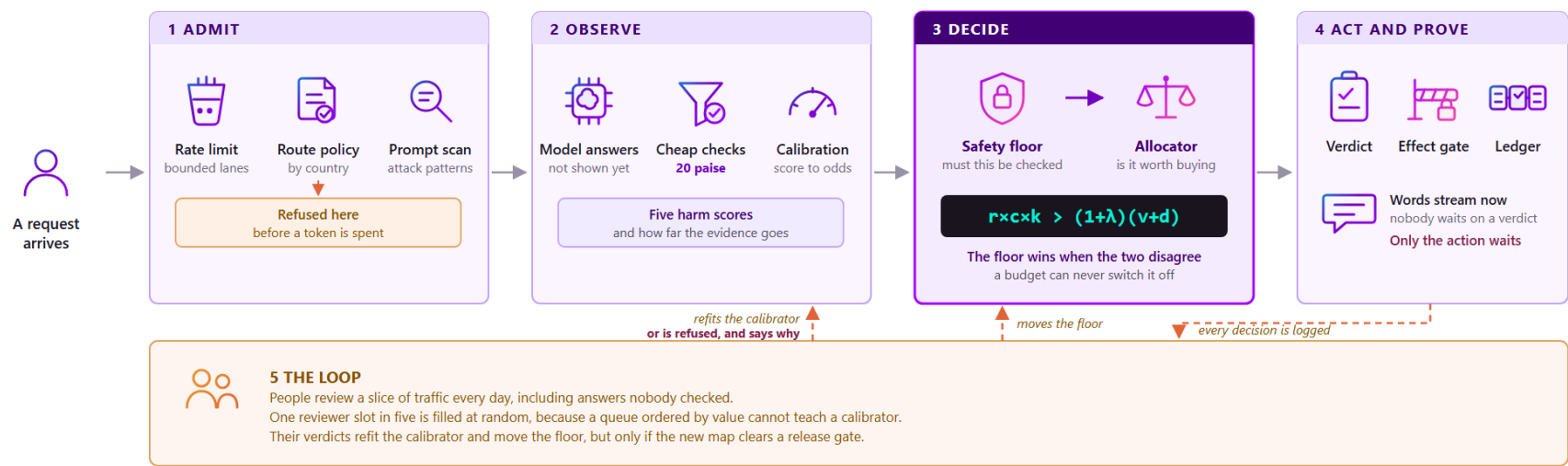
Words and actions are different risks

Text can stream while checking runs, because text can be retracted. A payment cannot, so effects wait behind their own gate.

04 Architecture

The gateway sees only the request, the response, any supplied context and any proposed tool calls. It needs no model weights, no hidden states and no log probabilities, so it works in front of a model an enterprise rents as well as one it hosts.

ControlPlane One gateway in front of any model. Every answer gets a priced decision and a receipt.



Stages one to four run on the request. Stage five runs overnight.
The gateway sees only what went in and what came out, so it works in front of a model an enterprise rents rather than owns.
Text is never delayed. Money and data are gated separately, and every decision is recomputable from its own record.

Stages one to four run on the request. Stage five runs on reviewed traffic. The diagram is hand authored SVG that regenerates from its own source, so it cannot drift away from the system it describes.

1 Admit	2 Observe	3 Decide	4 Act and prove	5 The loop
A per route token bucket and bounded lanes decide whether there is capacity, with capacity reserved for mandatory work. A preflight rules scan then reads the prompt alone and refuses it above an injection score of 0.70, before any token is generated. runtime/admission.py	Tier 0 rules and Tier 1 signals score the answer on five axes. The two merge, taking most of the loudest and some of the mean, so one confident detector is heard without a quiet one silencing it. Calibration then turns scores into probabilities. risk/calibration.py	The release floor marks what must be checked. The allocator prices each tier against the shadow price and picks the best net value. If Tier 2 is bought it runs and the verdict is taken again on the recomputed evidence. economics/allocator.py	A verdict of allow, annotate, abstain, hold or block covers the text. Proposed effects are permitted, held or denied independently. Everything is appended to the hash chain and anything undecided is queued for a person. ledger/	Reviewers label a slice of traffic. One slot in five is filled at random, because a queue ordered by value cannot teach a calibrator. Labels refit the maps only if the new artifact clears a release gate. learning/refit.py

05 End to end request flow

Held out row `cp-02477` . A finance route request carrying a `transfer_funds` call, where the model echoes back a secret the source document says must never be repeated. Every figure below is recomputed from the stored record.

#	Stage	What happened to this request
1	A request arrives with the route it belongs to	Route <code>finops-agent</code> , jurisdiction <code>india</code> , with a <code>transfer_funds</code> tool call attached.
2	Policy is resolved from the route and the country	Pack <code>india-dpdp-2025.11</code> , hash <code>a5adbea59457608b</code> . Exfiltration on this route is priced at ₹55,000. Financial effects always require a person.
3	The model answers and two cheap tiers read it for 20 paise	The answer attempts to bypass policy and repeat a secret taken from the supplied context. The source document states that value must never be echoed.
4	Scores become probabilities so the numbers can be priced	Prompt attack 1.000, hallucination 0.250, private data 0.150. Evidence regime is grounded, because a source document was supplied.
5	The safety floor is consulted before any price is considered	Route threshold 0.25 against a highest harm of 1.000. The check is mandatory. The budget is not allowed to skip this one.
6	Every tier is priced and the best net value wins	Expected loss on the table is ₹69,357. Tier 0 would prevent ₹45,667 for ₹0.02, Tier 1 ₹57,612 for ₹0.18, Tier 2 ₹63,123 for ₹3.20. Tier 2 is chosen.
7	A verdict is reached on the recomputed evidence	With the judge signal merged in, harm is 0.910, which sits above the 0.88 block boundary. The answer is blocked.
8	The action is gated apart from anything said in words	<code>transfer_funds</code> , classified financial, denied. The money never moves.
9	A receipt is written that cannot be edited later	All three tier sums, the threshold, the shadow price and the policy hash, chained to the record before it.

Stages five and six are the whole product. The floor decides whether a check is optional. The allocator decides which check is worth buying. Everything else is plumbing that keeps those two decisions honest and recomputable.

Note what stage seven shows. The judge did not simply confirm the cheap tiers. It moved the highest calibrated harm from 1.000 down to 0.910, which is a reduction, and the answer was still blocked because 0.910 remains above the block boundary. The second pass is a genuine recomputation rather than a rubber stamp.

06 Core decision logic

6.1 Tiered assurance

Three tiers sit behind one adapter contract. Each declares a cost, a latency and a per axis catch rate, and nothing else about a tier reaches the allocator.

Tier	What it is	Cost	Latency	Cost multiple	Catch, leak	Catch, injection
0	Rules, patterns and sensitive disclosure scoring	₹0.02	4 ms	1x	0.82	0.70
1	Lexical and grounding signals	₹0.18	70 ms	9x	0.90	0.84
2	Language model judge	₹3.20	900 ms	160x	0.94	0.91

From `config/economics.yaml`. Catch rate shown for the two axes that drive most decisions. Cost multiple is relative to Tier 0.

Catch rates are not left as configuration. `feedback/recalibration.py` estimates them from labelled outcomes. On the held out run **Tier 2 measured 0.930 against 0.880 configured over 365 observations**, and Tier 1 measured 0.788 against 0.680 over 48. A better detector therefore reports its own higher catch rate, and the allocator automatically buys more of it.

6.2 The release floor

The floor answers one question before any money is discussed. Is this answer risky enough that a check is obligatory. The threshold is not chosen by hand. It is selected on a held out fold using a learn then test procedure with an exact binomial upper bound over a grid of candidate thresholds, with the confidence budget split across the grid so that testing many candidates does not inflate the result.

Route	Threshold	Rows released	Escaped harm	Observed rate	Upper bound	Mandatory coverage
<code>finops-agent</code>	0.25	339	19	0.0560	0.0756	32.2%
<code>internal-kb</code>	1.00	476	34	0.0714	0.0890	4.8%
<code>support-assistant</code>	0.10	397	21	0.0529	0.0703	20.6%

Floor validation on 1,500 held out interactions, alpha 0.15 at 90% confidence. Source `docs/results/results.json`.

Takeaway. Every route holds well inside the bound, and holds non vacuously, meaning rows really were released unchecked and the bound still bound. The floor is only informative when alpha exceeds the harm base rate of the traffic. An operator whose traffic is one third harmful at alpha 0.15 is not getting a guarantee, they are getting full coverage, and should either raise alpha or budget for checking everything.

6.3 Budget allocation

Below the floor, the allocator compares tiers on net value, which is the expected loss a tier would prevent minus its cost inflated by the shadow price. The shadow price is the Lagrange dual of the budget constraint and behaves like a thermostat. It rises when observed spend runs ahead of the allowance and falls when it runs behind, so the cut line moves with live traffic while the budget itself stays where the operator put it.

A shadow price alone cannot bound spend, because it is a soft penalty and cannot price out a mandatory check. Ours did not bound it. At the tightest setting the allocator spent 3.75 times its budget. `BudgetGovernor` fixes this

by reserving the floor cost, so discretionary spending stops once committed spend plus expected remaining floor cost reaches the budget, and the allocator degrades to mandatory only rather than overspending.

1.00x to 1.03x

spend against budget across the whole grid, once governed

₹51.84

what the floor alone obliges, 1.08% of full coverage

5.625%

below this budget fraction, blanket cheap coverage stops being affordable

Two numbers make a budget honest and both are computed rather than asserted. The floor cost is the assurance bill the guarantee sends whatever the budget says, so a budget below it is infeasible and is reported as such rather than silently breached. The blanket coverage boundary is simply Tier 1 cost divided by Tier 2 cost. Above it, checking every row at Tier 1 is affordable and no selective policy can beat blanket coverage.

6.4 Human review

Cases the system will not decide alone go to a queue with fixed capacity. Because capacity saturates, the only lever left is order. The shipped rule serves by expected loss per reviewer minute weighted against the deadline.

Serving rule	Expected loss served	Deadlines missed	High value cases shed
deadline_density, shipped	₹3,430,681	65	2
density, ablation	₹3,798,647	48	2
random, baseline	₹2,816,476	47	9
deadline, ablation	₹2,200,133	150	15
fifo, baseline	₹2,162,224	139	15

Reviewer queue at the 10% budget, identical cases, 166 completed reviews in every row. Source [docs/results/attention.json](#).

Takeaway. Order alone moves 1.59x more expected loss through the same reviewer hours, and cuts high value cases dropped from 15 to 2. The **density** ablation, which is the shipped rule with the deadline term removed, leads on both axes and is reported as the stronger rule rather than hidden. Ordering remains the smaller lever. Keeping up with arrivals at this budget needs 3.0 reviewers against the two the scenario staffs.

6.5 Effect gating

A verdict is two decisions rather than one. Whether the words may be shown, and whether the proposed actions may run. Tool calls are classified by reversibility and blast radius into read, reversible write, irreversible write, external facing and financial. Each is then permitted, held or denied on its own, and a policy can require a person for whole effect classes regardless of score. On the finance route financial and irreversible effects always require a person.

What this is not. The gate returns a decision and writes it to the chain. It does not hold a durable lease over the effect. A gateway that crashes between deciding hold and the caller acting on it leaves no lock behind. That is enough to evaluate whether the decision is correct, priced and auditable. It is not enough to run irreversible effects against without adding a fencing token and restart recovery, and this is recorded in the code itself rather than only in the limitations document.

07 Audit and reproducibility

An assurance system that cannot show its working is an opinion. Two mechanisms make this one checkable, one at runtime and one at build time.

The ledger

Every decision is appended to a SQLite chain where each record carries the hash of the record before it. Change any earlier row and every hash after it stops matching. The record holds the calibrated harm vector, the raw pre calibration vector, the tier by tier arithmetic, the threshold applied, the shadow price, the effect actions and the policy version and hash. That is enough to recompute the decision rather than merely recall that it happened.

1,500 of 1,500 decisions recorded	224 of 224 proposed effects logged	205 reviews in the same chain	Valid chain verification over 1,705 records
---	--	---	---

The regeneration path

Every published figure is written by a committed command into a tracked file. `./run_submission.sh` creates a clean virtual environment, runs the quality gate, regenerates the demo and the report, and then checks that every regenerated artifact is byte identical to the committed one. If a number stops reproducing the script fails and names the files that changed.

Command	What it regenerates
<code>make report</code>	Allocation sweep, release floor validation, calibration error, audit completeness
<code>make attention</code>	Reviewer queue comparison across five serving rules
<code>make pii-probe</code>	Sensitive disclosure detection with per mechanism ablations
<code>make sensitivity</code>	Consequence price sweep across a 0.25x to 4x band
<code>make loadtest</code>	Admission control under offered load
<code>make relearn</code>	Calibration refit from reviewer labels, or a refusal with its reason
<code>make benchmarks</code>	Aegis, OR-Bench, BeaverTails and RAGTruth, each corpus pinned and checksummed

Registered endpoints

Each significant claim has a criterion written before the work started, stating what would have counted as success. Results are reported against those criteria whether or not they were met. The record includes a queue model defect that was written down before it was corrected, and a case where fitting a detector on the rows that later certified its bound made the bound claim 0.1407 while held out data showed 0.2800. Restoring the fold split fixed it. The discipline is the guarantee, and both halves of that are on the record.

08 Implementation

78 source modules under `controlplane/`, checked by `ruff`, `mypy --strict` and 191 tests across 28 files, all in one gate.

How the code is kept honest

Four mechanisms exist because this project has already been wrong in each of these ways, and each is enforced by a test rather than by intention.

- **Fold discipline.** No row used to fit a calibration map may be used to select the threshold that certifies it. Breaking this once made a released bound claim 0.1407 while held out data showed 0.2800.
- **Detector fingerprinting.** A calibration artifact records the detector stack it was fitted against and refuses to load against a different one, because serving a mismatched map corrupts every probability while total spend barely moves. Measured here, a cruder map changed 22.4% of tier decisions and removed all 80 blocks while spend stayed within 0.3%.
- **Byte identity.** `./run_submission.sh` regenerates every published artifact and fails if one differs from the committed copy.
- **Chain verification.** The ledger verifies end to end across both record shape changes this project has made, so chains written by an older version still parse and still verify.

One implementation detail worth reading

The sensitive disclosure scorer is the clearest example of the project's approach. A pattern matching detector for personal data scored 0.5881 AUC, and measuring the ceiling explained why. A *perfect* shape only detector reaches just 0.5869 on this corpus, because 309 held out rows carry a real identifier inside a permitted disclosure, such as an agent reading a customer their own work address, while 57 genuine leaks contain no recognisable pattern at all. Scoring whether a disclosure is *grounded in the authorised source* changes the result completely.

Detector	AUC	Precision	Recall	Rows flagged
Pattern rules	0.5881	0.11	n/a	n/a
Perfect shape only detector, the ceiling	0.5869	n/a	n/a	n/a
Microsoft Presidio	0.5825	0.0747	0.830	1,044
ControlPlane	0.9879	1.000	0.766	72

Presidio answers whether text contains personal data, and answers it accurately. The question a business needs answered is whether this requester may receive this value on this route, and that answer lives in the evidence rather than in the words.

Sensitive disclosure detection on 1,500 held out rows with 94 positives. Source `docs/results/pii.json`.

What is built

Capability	What is actually built	Module
Budget aware assurance	Expected loss against priced cost, with a shadow price controller and a governor that reserves the floor cost	<code>economics/</code>
Route specific safety floor	Learn then test threshold selection with an exact binomial upper bound and a disjoint fitting fold	<code>guarantees/conformal.py</code>

Capability	What is actually built	Module
Tiered verification	Three tiers behind one adapter contract, plus working adapters for a local language model judge and Microsoft Presidio	detectors/
Calibration	Isotonic regression by pool adjacent violators, interpolated between block boundaries, fitted per route and per axis	risk/calibration.py
Human review prioritisation	Five serving rules including the shipped one, with arrival times, deadlines, shedding and a random sampling reserve	review/queue.py
Effect gating	Effect classification by reversibility and blast radius, gated independently of the text verdict	effects/
Audit ledger	Hash chained SQLite records with immediate transactions and whole chain verification	ledger/
Admission control	Per route token buckets, bounded lanes, capacity reserved for mandatory work, and graceful degradation	runtime/admission.py
Learning loop	Joins raw scores in the ledger to reviewer labels, refits, reselects thresholds, and gates the release	learning/refit.py
Gateway	An OpenAI shaped chat completions API with streaming, so the plane drops in front of an existing client	gateway/app.py

09 Evaluation and evidence

1,500 held out interactions across three routes and two jurisdictions, with span level ground truth. Four policies compared across six budgets.

Allocation against the alternatives

Policy	Compute spend	Loss averted	Share of benefit	Share of cost	Escaped harm
Check nothing	₹0.00	₹3,184,800	58.2%	0.0%	0.0962
Allocator at 10% budget	₹480.98	₹5,078,000	92.8%	10.0%	0.0363
Allocator at 25% budget	₹1,098.58	₹5,407,000	98.9%	22.9%	0.0211
Allocator at 80% budget	₹3,839.64	₹5,477,300	100.1%	80.0%	0.0117
Check everything	₹4,800.00	₹5,469,400	100.0%	100.0%	0.0120

Loss averted and compute spent, 1,500 held out interactions. Benefit share is measured against checking every answer. Source [docs/results/results.json](#).

Takeaway. At a 10% budget the allocator captures 92.8% of the protection for 10.0% of the compute. At 80% it does not merely approach checking everything, it beats it, averting more loss for 20% less compute, because checking everything spends its budget on rows where the judge had nothing to find.

Calibration and price sensitivity

Expected calibration error runs from 0.008 to 0.043 by route, so the probabilities the money arithmetic multiplies are close to the frequencies observed. The consequence prices are the softest input in the system, so they were swept deliberately.

0.008 to 0.043

expected calibration error across the three routes

10.9%

of tier choices change across a 0.25x to 4x price swing

0%

of verdicts flip across that same swing

Takeaway. Getting the consequence prices wrong costs money, not safety. Price enters what a check is worth buying, and it does not enter the release rule. That separation is what makes the floor defensible to a risk owner who disagrees with finance about the numbers.

Behaviour under load

Under overload the choice is between slowing everyone down and refusing some people. Bounded admission takes the second, which keeps the tail flat and makes the refusal visible.

Policy	Text p99	Effect p99	Achieved rate	Rejected
Unbounded	540.6 ms	1508.1 ms	198.4	0
Bounded admission	26.2 ms	104.1 ms	63.7	499

At 400 offered requests per second. These figures measure the scheduler harness rather than a production model, because the detectors are stubs. Source [docs/results/runtime.md](#).

Read the last two columns before the first two. Bounded admission is not free. It turns 499 requests away and its throughput is a third of unbounded. What it buys is that the requests it does serve are served properly and the ones it cannot serve are told so, rather than everybody quietly waiting half a second. Under degradation every served request still completed its mandatory assessment, none bought a Tier 2 check, and no rejected request ever generated text.

When allocation is worth having, and when it is not

Budget aware allocation is not universally better than a well ranked fixed rate policy, and saying otherwise would not survive a judge with the repository open. At matched actual spend the allocator wins 5 of 7 budgets on our corpus, 4 of 7 on BeaverTails at a corrected prevalence, 3 of 7 at its natural prevalence, 3 of 7 on OR-Bench and 1 of 7 on Aegis, with gains between minus 20% and plus 11%.

Two conditions have to hold, and both are computable from the cost model and the label structure before anything is deployed. The harm mix must vary, so that equally risky rows carry unequal consequences. And blanket coverage at the cheapest effective tier must be unaffordable, meaning the budget sits below the 5.625% boundary. Aegis is the first public corpus where the first condition holds, and the allocator still wins only 1 of 7 budgets there, because the second condition fails.

10 External benchmarks

Five public corpora, none of them ours to grade, each with a criterion registered before it was run. Read the null column before the score. On an imbalanced corpus a policy that flags every row can look respectable while detecting nothing, and the margin over that null is the only part that is ours.

Benchmark	ControlPlane	Flag everything null	Margin	Published comparison	Verdict
ToxicChat, AUPRC	0.597	0.071	+0.526	Llama Guard 0.664, OpenAI Mod 0.588, Perspective 0.532	In band
RAGTruth, F1	0.601	0.518	+0.083	LettuceDetect 0.792, GPT-4 prompt 0.634, RAGAS 0.520	In band
BeaverTails at natural 55.8% harm, F1	0.747	0.716	+0.031	None published on our label mapping	Margin only
BeaverTails at corrected 7% harm, F1	0.165	0.139	+0.026	None published on our label mapping	Margin only
Aegis, AUPRC	0.811	0.661	+0.151	Band 0.860 to 0.941	Below band, failed
OR-Bench, AUC	0.784	0.500	+0.284	Operating point compared below	Partial

Source [docs/results/benchmarks.md](#) and the per corpus JSON files.

The floor on real traffic

The release floor was never violated on held out data. On five conditions it held non vacuously, meaning rows were genuinely released unchecked and the bound still bound. Observed unchecked harm ran from 0.0645 to 0.1156 against alpha 0.15, over populations from 525 to 9,994 released rows.

Elsewhere it held vacuously and that is reported as a limit rather than as more successes. On Aegis, OR-Bench and BeaverTails at its natural rate, the base rate exceeds alpha, so the floor demands a check on essentially every row, mandatory coverage reaches 1.0000, and the bound is satisfied by construction while carrying no information.

Aegis is a failure and is reported as one

0.811 AUPRC sits below Perspective API, the weakest published system on that benchmark. Two things it taught us are worth more than the score. Our lexical detectors flagged 100% of its rows while scoring an F1 that looks competitive purely because the base rate is 66%. And our disclosure mechanism drops from 0.9879 on our corpus to 0.5834 on Aegis, which is the sharpest available confirmation that it measures authorisation rather than recognition.

What we do to safe traffic

OR-Bench is the first measurement of over refusal. On prompts that state of the art models refuse despite being safe, ControlPlane refuses 13.5%, which is roughly a third of Llama-3-70b at 37.7%, while catching 66.7% of genuinely toxic prompts against its 78.7%. GPT-4o and Mistral-large beat us on both axes. Claude-3-Opus and Llama-3-70b do not.

The harness was validated before it was trusted. Our AUPRC for OpenAI Moderation on ToxicChat is 0.6321 against the published 0.588, a gap of 0.044 that we cannot fully account for and most likely reflects split or aggregation differences. It bounds how precisely any comparison on this page can be read.

11 Running the system

Python 3.11 or newer. The default path needs no API key, no network call, no model download and no GPU.

```
git clone https://github.com/Advait-Pathak9222/BurningShadows.git
cd BurningShadows
make install
make demo          # builds the corpus, calibrates, runs scenarios, verifies the chain
make prototype     # the live two panel prototype
make console       # the inspection console over the committed evidence
```

One command runs everything a reviewer would want, from a clean clone. It creates its own virtual environment, runs the quality gate, regenerates the demo and the report, and fails loudly if any committed artifact stops reproducing byte for byte.

```
./run_submission.sh
```

Two things to open

The live prototype puts a support chatbot next to the control plane, with the customer view on the left and every check, price and verdict on the right, revealed one step at a time. Three preset requests show a prompt injection stopped before the model runs at a cost of 2 paise, an ordinary lookup the allocator declines to spend the judge on, and a quiet data leak that the cheap tiers score below their own decision line and the judge catches.

The inspection console has six views over the committed evidence, covering the allocation sweep, the release floor, calibration, the reviewer queue and the ledger.

12 Deployment approach

The question is not whether ControlPlane is offline or API based. It is a hybrid, and deciding when to make the expensive call is the product.

Tier	Runs on	Where it belongs
0 Rules	100% of traffic	In process. A network hop here would cost more than everything it guards.
1 Small models	Most traffic	Usually in process. Self hosted embeddings are cheaper than a hosted API at enterprise volume.
2 Judge	A selected minority	This is where a hosted language model API belongs, and it is the call the allocator exists to ration.

The adapter path is built rather than hypothetical. `detectors/base.py` defines a single method contract, `ollama_judge.py` is a working adapter against a real local language model over HTTP, and `presidio_pii.py` adapts a real third party detector behind the same contract. Swap the detector, supply its three numbers, and the arithmetic is unchanged.

For part of the market offline is a hard requirement rather than a limitation. Banking and insurance under data localisation direction, anything where the disclosure itself is the regulated event, and healthcare, defence and air gapped deployments. Sending a suspected data leak to an external judge is itself an act of disclosure.

13 Current validation boundaries

Being straight about this is more useful than claiming readiness. The following is what a production deployment would have to close.

Boundary	Severity	What it means and what closes it
Calibration is fitted on synthetic traffic	High	The isotonic maps and the per route thresholds do not transfer to a different traffic distribution. Closing it needs roughly 500 labelled rows per route, which is about four weeks of one reviewer.
Drift breaks the bound silently	High	The finite sample guarantee is valid only for the distribution it was calibrated on. Production needs a drift detector that triggers recalibration. Not built.
Tier 2 is a deterministic stand in	Medium	The adapter exists and two real adapters are written against it. A production judge must be built and its catch rate re measured, which the feedback loop already handles.
Single process state	Medium	The ledger and the budget controller assume one process. Multiple workers need shared state.
The effect gate has no durable lease	Medium	It decides and records. It does not hold a lock across a crash. Irreversible effects need a fencing token and restart recovery.
Consequence prices need finance sign off	Low	Sensitivity across a 0.25x to 4x band moves 10.9% of tier decisions and flips 0% of verdicts, so getting this wrong costs money rather than safety.
The learning loop has not released	Low	73 usable labelled pairs against roughly 100 per route needed. A live desk accumulates labels across days. A single pass over 1,500 rows does not.

One measurement worth flagging on its own. Most reviewer labels cannot be learned from. The queue serves by expected loss per reviewer minute, so harmful rows are likelier to be reviewed within the raised population. That is selection happening inside a stratum and no stratum level weight undoes it. The value ordered sample runs 37.2% harmful against 18.4% in the traffic it has to price, and inverse probability weighting moved that only to 38.2%. This is why one reviewer slot in five is filled uniformly at random, and why that reserve is charged to every queue rule in the comparison. A serving figure measured without it is one no deployment can reach.

14 Conclusion

ControlPlane demonstrates that verification of AI output can be run as a priced decision rather than a fixed rule. Doing so is worth roughly 90% of the protection of checking everything for a tenth of the compute, while holding a safety bound the budget cannot switch off and producing a record that can be recomputed rather than trusted.

It also demonstrates the limits of that claim honestly. Allocation helps under conditions that are computable in advance and does not help outside them. The detectors are stubs, which is what makes the allocation result attributable and is also the first thing a deployment would replace. The reviewer queue, not the compute budget, is where the money and the risk actually sit.

The next step is the one the evidence points at. Take one route in one enterprise, run the gateway in shadow mode beside the existing stack, put a real judge behind the Tier 2 adapter, and refit calibration on that route's own labelled traffic until the learning loop stops refusing to release. Everything needed to do that is in this repository.